API Documentation

AI-Powered Energy Consumption Optimization Assistant

Local base URL: http://127.0.0.1:8000 **Interactive API:** http://127.0.0.1:8000/docs

Frontend URL: http://localhost:8501 (when the Streamlit frontend is running)

Documentation convention

Each API entry below uses the six required fields: Endpoint, HTTP Method, Request Format, Response Format, Sample Request, and Sample Response. Request/response examples are illustrative JSON examples based on the route and schema behavior; returned analysis content and retrieved records depend on runtime data. Fields not defined by the request schema are not presented as required inputs.

1. Service Information

Root / Service Information

Endpoint: /

HTTP Method: GET

Request Format: No request body or query parameters.

Response Format: JSON object containing the API message, configured application version, and docs path.

Sample Request:

```
GET /
```

Sample Response:

```
{
  "message": "AI-Powered Energy Consumption Optimization Assistant API",
  "version": "<configured APP_VERSION>",
  "docs": "/docs"
}
```

Notes: The version value is supplied by application configuration.

Health Check

Endpoint: /api/health

HTTP Method: GET

Request Format: No request body or query parameters.

Response Format: JSON object with service status and service name.

Sample Request:

```
GET /api/health
```

Sample Response:

```
{
  "status": "healthy",
  "service": "Energy Optimization Assistant"
}
```

2. Analysis and Retrieval APIs

Analyze Energy Scenario

Endpoint: /api/analyze

HTTP Method: POST

Request Format: JSON object validated by AnalyzeRequest. Required: query (string, 5–500 characters). Optional: top_k (integer 1–20, default 5), season, period, consumption_level, is_anomaly, baseline_kwh (>=0), tariff_per_kwh (>=0), emission_factor_kg_per_kwh (>0), reduction_rate (0–100), period_label (string, default 'specified period', max 100).

Response Format: JSON analysis result returned by the coordinator. Exact analysis fields depend on the coordinator workflow and supplied scenario.

Sample Request:

```
{
  "query": "How can I reduce high energy use during peak hours?",
  "top_k": 5,
  "period": "peak",
  "consumption_level": "high",
  "baseline_kwh": 120,
  "tariff_per_kwh": 0.15,
  "emission_factor_kg_per_kwh": 0.7,
  "reduction_rate": 10,
  "period_label": "daily"
}
```

Sample Response:

```
{
  "illustrative": true,
  "note": "The actual JSON fields and values are returned by the analysis coordinator and depend on the current
 implementation and data."
}
```

Notes: Use the live Swagger response to capture an exact response example for the current coordinator implementation.

Search Historical Cases

Endpoint: /api/search

HTTP Method: POST

Request Format: JSON object validated by SearchRequest. Required: query (string, 2–500 characters). Optional: top_k (integer 1–20, default 5), season, period, consumption_level, is_anomaly.

Response Format: On success: success, validated query, results array, and count. On guardrail rejection: success=false and error message.

Sample Request:

```
{
  "query": "high energy consumption during peak hours",
  "top_k": 5,
  "period": "peak",
  "consumption_level": "high"
}
```

Sample Response:

```
{
  "success": true,
  "query": "high energy consumption during peak hours",
  "results": ["<retrieved case objects depend on the vector store>"],
  "count": 5
}
```

Notes: Illustrative results array shown as a placeholder; actual entries are retrieved case objects. Rejected-query response format: {"success": false, "error": ""}.

3. Recommendation APIs

LLM-Assisted Recommendation

Endpoint: /api/recommend/llm

HTTP Method: POST

Request Format: JSON object validated by RecommendationRequest. Required: query (string, 5–500 characters). Optional: top_k (integer 1–20, default 5), season, period, consumption_level, is_anomaly, baseline_kwh (>=0), tariff_per_kwh (>=0), emission_factor_kg_per_kwh (>0), reduction_rate (0–100), period_label (default 'specified period', max 100).

Response Format: JSON recommendation result returned by coordinator.recommend_llm; exact fields depend on the current coordinator implementation and model response.

Sample Request:

```
{
  "query": "Suggest ways to reduce high consumption during peak hours",
  "top_k": 5,
  "period": "peak",
  "consumption_level": "high",
  "baseline_kwh": 120,
  "tariff_per_kwh": 0.15,
  "emission_factor_kg_per_kwh": 0.7,
  "reduction_rate": 10,
  "period_label": "daily"
}
```

Sample Response:

```
{
  "illustrative": true,
  "note": "See the live endpoint response for the exact recommendation schema and values."
}
```

Notes: Model availability and configuration can affect the result; do not treat estimates as measured savings.

Rule-Based Recommendation

Endpoint: /api/recommend/rule

HTTP Method: POST

Request Format: JSON object validated by RecommendationRequest. Required: query (string, 5–500 characters). Optional: top_k (integer 1–20, default 5), season, period, consumption_level, is_anomaly, baseline_kwh (>=0), tariff_per_kwh (>=0), emission_factor_kg_per_kwh (>0), reduction_rate (0–100), period_label (default 'specified period', max 100).

Response Format: JSON recommendation result returned by coordinator.recommend_rule; exact fields depend on the current coordinator implementation.

Sample Request:

```
{
  "query": "Suggest ways to reduce high consumption during peak hours",
  "top_k": 5,
  "period": "peak",
  "consumption_level": "high",
  "baseline_kwh": 120,
  "tariff_per_kwh": 0.15,
  "emission_factor_kg_per_kwh": 0.7,
  "reduction_rate": 10,
  "period_label": "daily"
}
```

Sample Response:

```
{
  "illustrative": true,
  "note": "See the live endpoint response for the exact rule-based recommendation schema and values."
}
```

Notes: This endpoint selects the rule-based recommendation path.

4. Evaluation and Feedback APIs

Evaluate Query

Endpoint: /api/evaluate

HTTP Method: POST

Request Format: JSON object validated by EvaluationRequest. Required: query (string, 5–500 characters). Optional: top_k (integer 1–20, default 5).

Response Format: JSON evaluation result returned by evaluation_service.evaluate_query; fields depend on the configured evaluation service and available labels.

Sample Request:

```
{
  "query": "high energy consumption during peak hours",
  "top_k": 5
}
```

Sample Response:

```
{
  "illustrative": true,
  "note": "Exact evaluation fields depend on the current evaluation service and test data."
}
```

Notes: Evaluation results should be interpreted in context of the dataset and evaluation method.

Submit Feedback

Endpoint: /api/feedback

HTTP Method: POST

Request Format: JSON object validated by FeedbackRequest. Required: query (string, 5–500 characters), recommendation (string, minimum 1 character), feedback (string, minimum 1 character). Optional: rating (number 1–5), comment (string).

Response Format: JSON result returned by coordinator.submit_feedback; exact fields depend on the coordinator implementation.

Sample Request:

```
{
  "query": "How can I reduce high energy use?",
  "recommendation": "Shift flexible loads away from peak hours.",
  "feedback": "helpful",
  "rating": 5,
  "comment": "Clear and actionable."
}
```

Sample Response:

```
{
  "illustrative": true,
  "note": "See the live endpoint response for the exact feedback result schema."
}
```

Feedback Summary

Endpoint: /api/feedback/summary

HTTP Method: GET

Request Format: No request body or query parameters.

Response Format: JSON summary returned by coordinator.feedback_summary.

Sample Request:

```
GET /api/feedback/summary
```

Sample Response:

```
{
  "illustrative": true,
  "note": "Summary fields and values depend on stored feedback records."
}
```

Feedback Records

Endpoint: /api/feedback/records

HTTP Method: GET

Request Format: No request body or query parameters.

Response Format: JSON data returned by coordinator.feedback_records.

Sample Request:

```
GET /api/feedback/records
```

Sample Response:

```
{
  "illustrative": true,
  "note": "Returned records depend on feedback currently stored."
}
```

5. Running and Testing the APIs

Start the backend from the project root:

```
uvicorn src.main:app --reload
```

Open the interactive API page and verify each endpoint's exact response:

```
http://127.0.0.1:8000/docs
```

Example PowerShell health check:

```
Invoke-RestMethod http://127.0.0.1:8000/api/health
```

Request validation notes

• FastAPI/Pydantic validates JSON request bodies and returns a validation error for invalid field types or constraints.

• Search requests have a minimum query length of 2 characters; analysis, recommendation, evaluation, and feedback queries have a minimum length of 5 characters.

• top_k is constrained to 1–20 where available.

• reduction_rate is expressed as a percentage from 0 to 100.

• Optional cost and carbon values are scenario inputs; calculations based on them are estimates, not proof of actual savings.

• For endpoints whose return structure is produced by coordinator/service methods, use /docs to capture a real sample response from the current running code. The illustrative placeholders in this document are explicitly marked and are not asserted as exact runtime payloads.